LA MINUTE DE CODE

FORMATION: JAVASCRIPT DE A À Z

PARTIE 1

LA MINUTE DE CODE

PLAN DE LA FORMATION

JS

- Introduction à la formation
- Le fonctionnement de Javascript
- Préparation à la formation
- Où écrire du JavaScript
- Les variables
- Les types de valeurs
- Les opérateurs
- Templates literals
- Les fonctions
- La porté des variables
- Les sélecteurs
- Les événements
- Styliser des éléments du DOM

LA MINUTE DE CODE

PRÉSENTATION

JS

JavaScript est un langage de programmation de haut niveau, qui est interprété plutôt que compilé. Cela signifie que le code source est lu et exécuté ligne par ligne par le navigateur, plutôt que d'être pré-compilé en code machine comme dans d'autres langages tels que C ou C++.
JavaScript est également un langage orienté objet, ce qui signifie qu'il utilise des concepts tels que les classes, les objets, les méthodes et les propriétés pour organiser et structurer le code. Cela permet une plus grande modularité et une réutilisation du code.
JavaScript est principalement utilisé pour créer des applications web interactives. En utilisant JavaScript, vous pouvez ajouter des animations, des effets visuels et des interactions utilisateur sophistiquées à une page web, sans avoir besoin d'une rechargement de la page.

Les principales caractéristiques de JavaScript comprennent :

- Syntaxe simple : JavaScript utilise une syntaxe relativement simple qui est facile à lire et à comprendre.
- Flexibilité : JavaScript est un langage flexible qui peut être utilisé pour une grande variété de tâches, allant de la création d'applications web simples à la création d'applications plus complexes.
- Client-serveur : JavaScript peut être utilisé côté client (dans le navigateur web) ou côté serveur (avec Node.js). Cela permet aux développeurs de créer des applications web complètes en utilisant un seul langage de programmation.
- Événements : JavaScript permet aux développeurs de réagir aux événements de l'utilisateur, tels que les clics de souris, les entrées de clavier et les mouvements de la souris.
- API : JavaScript possède une grande variété d'API (interfaces de programmation d'applications) qui permettent aux développeurs d'accéder à des fonctionnalités supplémentaires telles que les API de géolocalisation, les API de dessin et les API de médias.

JavaScript est largement utilisé dans l'industrie du développement web et est un langage de choix pour de nombreux développeurs en raison de sa facilité d'utilisation et de sa flexibilité. Il est également constamment mis à jour et amélioré pour répondre aux besoins croissants des développeurs et des utilisateurs web.

LA MINUTE DE CODE

INFORMATIONS

Notre formation sur Javascript est conçue pour être accessible à tous, sans prérequis particulier. Que vous soyez débutant, en reconversion professionnelle ou simplement amateur, cette formation est faite pour vous.

Nous croyons que tout le monde devrait avoir la possibilité d'apprendre les compétences nécessaires pour créer des sites web professionnels et efficaces.

C'est pourquoi notre formation est adaptée à tous les niveaux de compétence, du débutant absolu à l'utilisateur avancé cherchant à acquérir de nouvelles compétences.

Cette formation vous guidera pas à pas à travers chaque étape, en utilisant des exemples pratiques et des exercices pour vous aider à appliquer ce que vous avez appris.

Vous aurez également accès à des ressources supplémentaires, telles que des forums de discussion et des documents de référence, pour vous aider à approfondir vos connaissances et à répondre à toutes vos questions.

Que vous cherchiez à vous lancer dans une carrière en développement web, à améliorer vos compétences existantes ou simplement à vous amuser en créant des sites web pour vous-même ou pour vos amis, notre formation sur Javascript est la solution idéale pour vous.

Rejoignez-nous dès maintenant pour découvrir comment créer des sites web professionnels et attrayants avec Javascript, sans prérequis requis. Nous sommes impatients de vous aider à atteindre vos objectifs de développement web.

PRÉPARATION

Tout ce dont vous avez besoin pour commencer à apprendre le Javascript est un ordinateur, une connexion Internet et un éditeur de code.

Un éditeur de code est un logiciel qui vous permet d'écrire et de modifier du code Javascript. Il existe de nombreux éditeurs de code disponibles gratuitement sur Internet, tels que Sublime Text, Visual Studio Code, Notepad++, Atom, entre autres.

Sublime Text : éditeur de code populaire et largement utilisé par les développeurs web. Il est facile à utiliser, rapide et dispose de fonctionnalités avancées telles que la coloration syntaxique, la complétion automatique, la gestion de projet et la prise en charge de plugins. https://www.sublimetext.com/

Visual Studio Code : éditeur de code open source développé par Microsoft. Il offre des fonctionnalités avancées telles que la coloration syntaxique, la complétion automatique, la débogage, la prise en charge de Git et des plugins. https://code.visualstudio.com/

Notepad++ : éditeur de code gratuit pour Windows, qui est simple et facile à utiliser. Il offre des fonctionnalités telles que la coloration syntaxique, la numérotation des lignes et la recherche et remplacement de texte. https://notepad-plus-plus.org/

Atom : éditeur de code open source développé par GitHub. Il offre des fonctionnalités avancées telles que la coloration syntaxique, la gestion de projet, la complétion automatique et la prise en charge de plugins. https://atom.fr.uptodown.com/windows

LA MINUTE DE CODE

FONCTIONNEMENT

JS

JavaScript est un langage de programmation qui est exécuté dans un environnement web, généralement un navigateur web. Lorsque vous visitez une page web contenant du code JavaScript, votre navigateur télécharge et exécute ce code pour créer une expérience utilisateur interactive.

Le processus de fonctionnement de JavaScript est le suivant :

1. Téléchargement : Tout d'abord, lorsque vous visitez une page web, le navigateur web télécharge le code HTML, CSS et JavaScript associé à la page.
2. Interprétation : Ensuite, le navigateur interprète le code JavaScript en utilisant son moteur d'exécution JavaScript intégré. Les moteurs d'exécution JavaScript les plus courants sont V8 (utilisé dans Chrome) et SpiderMonkey (utilisé dans Firefox).
3. Création d'un modèle de document : Le navigateur crée un modèle de document, également appelé "Document Object Model" (DOM), qui représente la page web. Le DOM est une structure arborescente qui représente les éléments HTML de la page, ainsi que les relations hiérarchiques entre eux.
4. Manipulation de l'interface utilisateur : Le code JavaScript peut ensuite être utilisé pour manipuler les éléments HTML de la page, ce qui permet de créer une interface utilisateur interactive. Par exemple, le code JavaScript peut être utilisé pour ajouter ou supprimer des éléments HTML, modifier le style CSS des éléments, ou répondre aux événements utilisateur tels que les clics de souris ou les entrées de clavier.
5. Communication avec le serveur : Le code JavaScript peut également être utilisé pour communiquer avec un serveur web, en utilisant des requêtes AJAX ou en utilisant des techniques plus avancées telles que les WebSockets.

En somme, JavaScript fonctionne en permettant aux développeurs d'écrire du code qui peut interagir avec les éléments HTML de la page web, ainsi qu'avec d'autres technologies web telles que le CSS et le serveur web. Cela permet de créer des pages web dynamiques et interactives qui offrent une expérience utilisateur améliorée.

OU ÉCRIRE DU JAVASCRIPT

Il y a deux façons principales d'intégrer du code JavaScript dans une page HTML :

Lien externe : cette méthode consiste à créer un fichier JavaScript séparé et à le lier à la page HTML à l'aide d'un élément script. Voici un exemple de code HTML :

```html
<!DOCTYPE html>
<html>
<head>
  <title>Ma page</title>
</head>
<body>
  <h1>Titre</h1>
  <p>Paragraphe</p>
  <script src="script.js"></script>
</body>
</html>
```

Dans cet exemple, le fichier JavaScript script.js est lié à la page HTML à l'aide de la balise script. Tout le code JavaScript défini dans le fichier sera exécuté lorsque la page HTML est chargée.

Code en ligne : cette méthode consiste à ajouter du code JavaScript directement à la page HTML à l'aide de la balise script. Voici un exemple de code HTML :

LA MINUTE DE CODE

LES VARIABLES

JS

En JavaScript, les variables sont des symboles qui représentent une valeur. Une variable est un conteneur dans lequel on peut stocker des données.

Elles peuvent être utilisées pour stocker des nombres, des chaînes de caractères, des objets, des fonctions, des tableaux et d'autres types de données.

En JavaScript, les variables sont déclarées à l'aide du mot-clé "var", "let" ou "const". Voici un exemple de déclaration de variable :

```
var prenom = `Jonathan`;
let prenom2 = 'Marie';
const prenom3 = "Nelya";
```

Les variables peuvent également être utilisées pour stocker des valeurs de retour de fonctions ou pour stocker des valeurs d'entrée fournies par l'utilisateur.

Il est important de noter que les variables en JavaScript sont dynamiques, ce qui signifie qu'elles peuvent être de n'importe quel type et que leur type peut changer à tout moment. Par exemple, une variable qui contient un nombre peut être modifiée pour contenir une chaîne de caractères ou un objet.

En utilisant des variables, les développeurs peuvent stocker et manipuler des données de manière dynamique dans leurs applications JavaScript, ce qui rend leur code plus flexible et évolutif.

LES TYPES DE VALEURS

En JavaScript, il existe plusieurs types de valeurs que l'on peut stocker dans des variables. Voici les principaux :

Les nombres : Les nombres sont des valeurs numériques, que ce soit des entiers ou des nombres à virgule flottante.

```
// Number
let nb = 2;
let nb2 = 2.5;
```

Les chaînes de caractères : Les chaînes de caractères sont des valeurs qui représentent une séquence de caractères. Elles sont délimitées par des guillemets simples ou doubles.

```
// string
let txt = 'Jonathan';
let txt2 = "Jonathan";
let txt3 = `Jonathan`
```

Les booléens : Les booléens sont des valeurs qui représentent vrai (true) ou faux (false).

```
// Boolean
let bool = false;
```

Les tableaux : Les tableaux sont des structures de données qui permettent de stocker une liste de valeurs. Les éléments du tableau peuvent être de n'importe quel type de valeur, y compris d'autres tableaux.

```
// tableau
let tab = [1,2,3,4,5]
let tab2 = ['Jonathan', 'Marie',"Nelya"]
```

LA MINUTE DE CODE

LES SELECTEURS

JS

En JavaScript, les sélecteurs sont utilisés pour cibler des éléments HTML spécifiques dans une page web. Les sélecteurs permettent aux développeurs d'appliquer des styles CSS, de manipuler le contenu et d'ajouter des événements à ces éléments HTML. Voici quelques-uns des sélecteurs les plus couramment utilisés en JavaScript :

Sélecteur par ID : Le sélecteur par ID permet de cibler un élément HTML spécifique en utilisant son attribut ID. Pour cibler un élément avec un ID spécifique, on utilise la fonction document.getElementById('id').

```
const titre3 = document.querySelector('#nomDeMonId');
const tile = document.getElementById('title')
```

Sélecteur par nom de balise : Le sélecteur par nom de balise permet de cibler tous les éléments HTML d'un type spécifique en utilisant le nom de la balise. Pour cibler tous les éléments d'une balise spécifique, on utilise la fonction document.getElementsByTagName('tag').

```
const el = document.getElementsByTagName('a')
```

Sélecteur par classe : Le sélecteur par classe permet de cibler tous les éléments HTML qui ont une classe spécifique en utilisant le nom de la classe. Pour cibler tous les éléments avec une classe spécifique, on utilise la fonction document.getElementsByClassName('class').

```
const titre2 = document.querySelector('.nomDeMaClasse');
const text = document.getElementsByClassName('texte')
```

LA MINUTE DE CODE

LES FONCTIONS

Une fonction est un bloc de code qui peut être appelé pour effectuer une tâche spécifique. Les fonctions sont utilisées pour diviser un programme en blocs de code plus petits et plus gérables, et pour réutiliser du code commun.

Les fonctions en JavaScript sont un élément clé de la programmation JavaScript et sont utilisées pour effectuer de nombreuses tâches différentes, notamment la manipulation de données, la gestion des événements et la création d'animations et de fonctionnalités interactives sur les pages web.

Fonction classique:

Les fonctions classiques en JavaScript sont également appelées fonctions déclaratives. Elles sont définies en utilisant le mot-clé function suivi du nom de la fonction, des parenthèses et éventuellement une liste de paramètres entre ces dernières, et enfin des accolades qui entourent le corps de la fonction. Voici un exemple de fonction classique en JavaScript :

```
function presentation(){
  console.log('hello');
}
```

Ensuite, une fois déclarée ont l'éxécute:

```
presentation()
```

LA MINUTE DE CODE

LES ÉVÉNEMENTS

JS

En JavaScript, les événements sont des actions qui se produisent sur une page web, comme un clic sur un bouton, la soumission d'un formulaire, le survol d'une image, etc.

Les événements sont déclenchés par l'utilisateur ou par le navigateur et sont utilisés pour déclencher des fonctions qui effectuent des actions spécifiques en réponse à l'événement.

Voici un exemple simple d'événement en JavaScript :

```
document.getElementById("myButton").addEventListener
("click", function() {
  alert("Hello World!");
});
```

Dans cet exemple, un événement "click" est attaché à un bouton HTML avec l'ID "myButton". Lorsque l'utilisateur clique sur ce bouton, la fonction anonyme est appelée et une boîte de dialogue "Hello World!" s'affiche.

Il existe de nombreux types d'événements en JavaScript, notamment :

1. Les événements de souris : Ces événements sont déclenchés par des actions de la souris, comme un clic, un double-clic, un survol, etc.
2. Les événements de clavier : Ces événements sont déclenchés par des actions du clavier, comme l'appui sur une touche, la libération d'une touche, etc.
3. Les événements de formulaire : Ces événements sont déclenchés par des actions sur un formulaire, comme la soumission du formulaire, le changement d'un champ de saisie, etc.
4. Les événements de fenêtre : Ces événements sont déclenchés par des actions sur la fenêtre du navigateur, comme le chargement de la page, le redimensionnement de la fenêtre, etc.

LA MINUTE DE CODE

LES ÉVÉNEMENTS

Pour gérer les événements en JavaScript, on peut utiliser la méthode "addEventListener" pour ajouter un gestionnaire d'événements à un élément HTML. Le gestionnaire d'événements est une fonction qui est appelée lorsque l'événement est déclenché, et qui effectue une action spécifique en réponse à l'événement.

En utilisant les événements en JavaScript, les développeurs peuvent créer des pages web interactives et réactives, qui répondent aux actions de l'utilisateur de manière dynamique et en temps réel.

LA MINUTE DE CODE

LES VARIABLES

JS

il y a trois mots-clés qui permettent de déclarer des variables : var, let et const. Ces trois mots-clés ont des comportements légèrement différents en termes de portée, de réaffectation et d'immutabilité.

Voici les principales différences entre les variables var, let et const :

1. Portée : La portée d'une variable détermine la zone de programme où la variable peut être utilisée. Les variables déclarées avec var ont une portée de fonction, ce qui signifie qu'elles sont accessibles à l'intérieur de la fonction dans laquelle elles sont déclarées. Les variables déclarées avec let et const ont une portée de bloc, ce qui signifie qu'elles ne sont accessibles qu'à l'intérieur du bloc (généralement délimité par des accolades) dans lequel elles sont déclarées.
2. Réaffectation : La réaffectation d'une variable consiste à lui donner une nouvelle valeur. Les variables déclarées avec var et let peuvent être réaffectées, ce qui signifie que leur valeur peut être modifiée au cours du temps. Les variables déclarées avec const, en revanche, ne peuvent pas être réaffectées. Cela signifie que leur valeur est définie une fois lors de leur déclaration et ne peut pas être modifiée par la suite.
3. Immutabilité : En plus de ne pas pouvoir être réaffectées, les variables déclarées avec const sont également immuables, c'est-à-dire que leur valeur ne peut pas être modifiée. Cela signifie qu'une variable const ne peut pas être redéfinie ou réaffectée. Les variables déclarées avec var et let ne sont pas immuables et peuvent être modifiées.

```
// ceci est faisable
prenom = 'Lucie'
prenom2 = 'Damien'
// ceci ne fonctionne pas. les
const sont des variables non
réasigniable
prenom3 = 'Elodie'
```

En résumé, voici les principaux avantages et inconvénients de chaque type de variable :

- var : Avantages - disponible dans toutes les versions de JavaScript, portée de fonction globale, peut être réaffectée ; Inconvénients - pas de portée de bloc, facilement sujet à des problèmes de portée, peut être réaffectée
- let : Avantages - portée de bloc, peut être réaffectée ; Inconvénients - pas disponible dans les versions antérieures de JavaScript, peut être sujet à des problèmes de portée
- const : Avantages - portée de bloc, immuable, peut aider à prévenir les bugs ; Inconvénients - pas disponible dans les versions antérieures de JavaScript, ne peut pas être réaffectée, ne peut pas être utilisée pour des valeurs qui changent fréquemment.

En général, il est recommandé d'utiliser const par défaut pour toutes les variables, sauf si vous avez une raison de réaffecter la variable. Si vous avez besoin de réaffecter une variable, utilisez let. Évitez d'utiliser var dans la mesure du possible car cela peut causer des problèmes de portée et d'autres bugs potentiels.

LA MINUTE DE CODE

LES OPÉRATEURS

JS

En JavaScript, les opérateurs sont utilisés pour effectuer des opérations sur les valeurs et les variables. Il existe plusieurs types d'opérateurs en JavaScript, notamment :

1. **Opérateurs arithmétiques** : ces opérateurs sont utilisés pour effectuer des opérations mathématiques, tels que l'addition (+), la soustraction (-), la multiplication (*), la division (/), le modulo (%), et l'exponentiation (**).
2. **Opérateurs d'assignation** : ces opérateurs sont utilisés pour attribuer une valeur à une variable, tels que l'opérateur d'assignation simple (=), l'opérateur d'assignation avec addition (+=), l'opérateur d'assignation avec soustraction (-=), l'opérateur d'assignation avec multiplication (*=), et ainsi de suite.
3. **Opérateurs de comparaison :** ces opérateurs sont utilisés pour comparer les valeurs et renvoyer une valeur booléenne (vrai/faux), tels que l'opérateur d'égalité stricte (===), l'opérateur d'inégalité stricte (!==), l'opérateur d'égalité (==), l'opérateur d'inégalité (!=), l'opérateur de supériorité (>), l'opérateur de supériorité ou égalité (>=), l'opérateur d'infériorité (<), et l'opérateur d'infériorité ou égalité (<=).
4. **Opérateurs logiques :** ces opérateurs sont utilisés pour combiner ou inverser des valeurs booléennes, tels que l'opérateur ET (&&), l'opérateur OU (||), et l'opérateur NON (!).
5. **Opérateurs de concaténation de chaînes de caractères :** ces opérateurs sont utilisés pour concaténer des chaînes de caractères, tels que l'opérateur de concaténation (+).
6. **Opérateurs ternaires :** cet opérateur est utilisé pour effectuer une condition ternaire, qui permet d'exécuter une instruction ou une autre en fonction d'une condition booléenne. L'opérateur ternaire est représenté par le symbole "?" suivi d'une instruction si la condition est vraie, suivi par ":" et une instruction si la condition est fausse.
7. **Opérateurs d'incrémentation et de décrémentation :** ces opérateurs sont utilisés pour augmenter ou diminuer la valeur d'une variable de 1, tels que l'opérateur d'incrémentation (++), et l'opérateur de décrémentation (--).

Ces opérateurs sont utilisés couramment en JavaScript pour manipuler les valeurs et les variables dans les programmes. Il est important de comprendre comment chacun de ces opérateurs fonctionne afin de les utiliser efficacement dans votre code.

LA MINUTE DE CODE

TEMPLATES LITERALS

Les templates literals en JavaScript sont une fonctionnalité introduite dans la version ES6 (ECMAScript 2015) qui permet de créer des chaînes de caractères plus facilement et de manière plus lisible.

Les templates literals sont définis à l'aide de la syntaxe des backticks ( ) au lieu des guillemets simples ou doubles. Cela permet d'inclure des expressions JavaScript à l'intérieur de la chaîne de caractères en utilisant la notation ${expression}.

Par exemple, avec les templates literals, nous pouvons facilement concaténer des variables et des chaînes de caractères. Voici un exemple :

```
const firstName = "Jean";
const lastName = "Dupont";
const message = `Bonjour, je m'appelle ${firstName} ${lastName}.`;
console.log(message);
```

Le code ci-dessus crée une variable message qui contient une chaîne de caractères contenant le prénom et le nom de famille de la personne.
Les templates literals permettent également d'insérer des sauts de ligne et des espaces dans les chaînes de caractères en utilisant les caractères spéciaux comme \n et \t.

Les templates literals sont une fonctionnalité utile de JavaScript qui simplifie la création de chaînes de caractères complexes et rend le code plus lisible et plus facile à maintenir.

LA MINUTE DE CODE

LES TYPES DE VALEURS

Les objets : Les objets sont des structures de données complexes qui permettent de stocker une collection de propriétés et de méthodes. Les propriétés sont des paires clé-valeur qui représentent des attributs de l'objet, tandis que les méthodes sont des fonctions qui peuvent être appelées sur l'objet.

```
// objet
let obj = {
  marque: 'Citroen',
  annee: 2015 }
```

Les fonctions : Les fonctions sont des blocs de code qui peuvent être appelés à partir d'autres parties de l'application. Elles peuvent prendre des paramètres et renvoyer des valeurs.

```
const op = (nb1, nb2) => nb1 + nb2;
```

Les valeurs null et undefined : Les valeurs null et undefined représentent l'absence de valeur. La valeur null est utilisée pour indiquer une valeur vide, tandis que la valeur undefined est utilisée pour indiquer une valeur non définie.

```
// undefined
let und;

// null
let nul = null;
```

Il est important de comprendre les différents types de valeurs en JavaScript, car cela permet aux développeurs de manipuler les données de manière appropriée et efficace dans leurs applications.

LA MINUTE DE CODE

OU ÉCRIRE DU JAVASCRIPT

```html
<!DOCTYPE html>
<html>
<head>
  <title>Ma page</title>
</head>
<body>
  <h1>Titre</h1>
  <p>Paragraphe</p>
  <script>
    alert("Bonjour !");
  </script>
</body>
</html>
```

Dans cet exemple, du code JavaScript est ajouté directement à la page HTML à l'aide de la balise script. Ce code sera exécuté lorsque la page HTML est chargée.

Il est important de noter que l'inclusion de scripts dans une page HTML doit être effectuée avec prudence. Le code JavaScript peut potentiellement affecter la performance de la page, en particulier si le code est lourd ou mal optimisé.

De plus, il peut y avoir des problèmes de sécurité si le code est malveillant ou exploitable. Il est donc recommandé de toujours valider et tester soigneusement le code JavaScript avant de l'inclure dans une page HTML.

LA MINUTE DE CODE

LES OPÉRATEURS

JS

Pour cette premier partie, nous allons nous concentrer sur les opérateurs arithmétiques avant de comprendre les autres opérateurs.

```
let nb1 = 5;
let nb2 = 2;

// addition
let nb3 = nb1 + nb2;

// soustraction
let nb4 = nb1 - nb2;

// multiplication
let nb5 = nb1 * nb2;

// division
let nb6 = nb1 / nb2;

// modulo = le reste d'une division
let nb7 = nb1 % nb2;

// si une valeur number et une valeur string et concatainés
// cela nous retournera une valeur de type string.
let nb8 = nb2 + "Jonathan";
```

LA MINUTE DE CODE

LES SELECTEURS

Sélecteur par sélecteur de CSS : Les sélecteurs de CSS permettent de cibler des éléments HTML en utilisant des sélecteurs CSS. Les sélecteurs CSS peuvent être utilisés pour cibler des éléments spécifiques en fonction de leur position, de leurs attributs, de leur contenu et de bien d'autres choses encore. Pour cibler des éléments en utilisant un sélecteur CSS, on utilise la fonction document.querySelector('selector') ou document.querySelectorAll('selector').

```
const items = document.querySelectorAll('li')
```

Sélecteur par attribut : Le sélecteur par attribut permet de cibler des éléments HTML en utilisant un attribut spécifique. Par exemple, on peut cibler tous les éléments qui ont un attribut spécifique en utilisant la fonction document.querySelectorAll('[attribute]').

```
const items = document.querySelectorAll('[name="choix"]')
```

En utilisant ces sélecteurs, les développeurs peuvent manipuler les éléments HTML dans leur application web de manière dynamique en utilisant du JavaScript.

LES FONCTIONS

Nous pouvons également ajouter des paramètres

```
function calculerSomme(a, b) {
  return a + b;
}
```

Cette fonction prend deux paramètres, a et b, et retourne leur somme.
Le mot-clé return est utilisé pour renvoyer une valeur à partir d'une fonction.
Dans l'exemple ci-dessus, la fonction retourne la somme des deux paramètres en utilisant return.
Voici un exemple d'appel de la fonction calculerSomme :

```
let resultat = calculerSomme(3, 5);
console.log(resultat); // Affiche 8
```

Dans cet exemple, la fonction calculerSomme est appelée avec les arguments 3 et 5. Le résultat de l'appel de la fonction est stocké dans la variable resultat qui est ensuite affichée à la console.

En résumé, les fonctions classiques en JavaScript permettent de définir des blocs de code réutilisables en utilisant le mot-clé function. Le mot-clé return permet de renvoyer une valeur à partir de la fonction.

LES FONCTIONS

Fonction fléché

Les fonctions fléchées en JavaScript sont une syntaxe plus concise pour définir des fonctions. Elles sont définies en utilisant une syntaxe plus simple que les fonctions classiques, avec une flèche ⇒ qui sépare les paramètres et le corps de la fonction.

Voici un exemple de fonction fléchée en JavaScript qui calcule la somme de deux nombres :

```
const calculerSomme = (a, b) => {
  return a + b;
}
```

Comme pour les fonctions classiques, le mot-clé return est utilisé pour renvoyer une valeur à partir de la fonction.

Les fonctions fléchées ont une syntaxe plus concise pour les cas simples où la fonction ne contient qu'une seule instruction, qui est également une instruction de retour. Dans ce cas, la fonction peut être écrite sur une seule ligne, comme ceci :

```
const calculerSomme = (a, b) => a + b;
```

L'appel de cette fonction est identique à l'appel d'une fonction classique :

```
let resultat = calculerSomme(3, 5);
console.log(resultat); // Affiche 8
```

LES FONCTIONS

Fonction callback

En JavaScript, les fonctions de rappel (callback) sont des fonctions qui sont passées en tant qu'argument à une autre fonction, et qui sont exécutées plus tard, généralement après une certaine opération asynchrone, telle qu'un appel à une API ou une requête de base de données.

Voici un exemple de fonction de rappel en JavaScript :

```
function faireQuelqueChose(callback) {
  // Faire quelque chose ici, puis exécuter la
fonction de rappel
  callback();
}

function maFonctionDeRappel() {
  console.log("La fonction de rappel a été exécutée
!");
}

// Passer maFonctionDeRappel en tant que fonction de
rappel à faireQuelqueChose
faireQuelqueChose(maFonctionDeRappel);
```

Dans cet exemple, la fonction faireQuelqueChose prend une fonction de rappel en tant qu'argument, puis exécute cette fonction plus tard en appelant simplement callback(). La fonction maFonctionDeRappel est passée en tant qu'argument à faireQuelqueChose, et elle est exécutée plus tard lorsque callback() est appelé.

LES FONCTIONS

Les fonctions de rappel peuvent également prendre des arguments, comme ceci :

```
function faireQuelqueChoseAvecDesDonnees(callback) {
  let donnees = "Les données que je veux passer à la fonction de rappel";
  callback(donnees);
}

function maFonctionDeRappel(donnees) {
  console.log("La fonction de rappel a été exécutée avec les données suivantes :", donnees);
}

// Passer maFonctionDeRappel en tant que fonction de rappel à faireQuelqueChoseAvecDesDonnees
faireQuelqueChoseAvecDesDonnees(maFonctionDeRappel);
```

Dans cet exemple, la fonction faireQuelqueChoseAvecDesDonnees prend une fonction de rappel en tant qu'argument, puis appelle cette fonction en passant les données en tant qu'argument.

En résumé, les fonctions de rappel en JavaScript sont des fonctions qui sont passées en tant qu'argument à une autre fonction, et qui sont exécutées plus tard, généralement après une opération asynchrone. Les fonctions de rappel peuvent également prendre des arguments.

LA MINUTE DE CODE

LA PORTÉ DES VARIABLES

La portée des variables en JavaScript détermine l'accessibilité de ces variables à l'intérieur du code. En d'autres termes, la portée des variables définit où la variable est disponible et où elle ne l'est pas.

Il existe deux types de portée des variables en JavaScript : la portée locale (aussi appelée portée de fonction) et la portée globale.
La portée locale est définie par les accolades d'une fonction. Toute variable déclarée à l'intérieur d'une fonction ne sera accessible qu'à l'intérieur de cette fonction, et pas à l'extérieur.

Par exemple :

```
function maFonction() {
  let maVariable = "Bonjour";
  console.log(maVariable); // Affiche "Bonjour"
}

maFonction();
console.log(maVariable); // Erreur : maVariable n'est
pas défini en dehors de la fonction
```

LA PORTÉ DES VARIABLES

Dans cet exemple, la variable maVariable est définie à l'intérieur de la fonction maFonction, ce qui la rend inaccessible à l'extérieur de cette fonction.

La portée globale, en revanche, est définie en dehors de toutes les fonctions. Toute variable déclarée à l'extérieur d'une fonction sera accessible à l'intérieur de toutes les fonctions et dans le code global.

Par exemple :

```
let maVariableGlobale = "Bonjour";

function maFonction() {
  console.log(maVariableGlobale); // Affiche "Bonjour"
}

maFonction();
console.log(maVariableGlobale); // Affiche "Bonjour"
```

Dans cet exemple, la variable maVariableGlobale est définie à l'extérieur de la fonction maFonction, ce qui la rend accessible à l'intérieur de cette fonction et dans le code global.

LA MINUTE DE CODE

LA PORTÉ DES VARIABLES

JS

Il est important de noter que les variables déclarées avec le mot-clé var ont une portée de fonction, tandis que les variables déclarées avec let et const ont une portée de bloc (portée locale).

Cela signifie que les variables var sont accessibles à l'intérieur de la fonction où elles sont définies, même si elles sont définies à l'intérieur d'un bloc if, for, ou while. Les variables let et const, en revanche, ne sont accessibles qu'à l'intérieur du bloc où elles sont définies.

En résumé, la portée des variables en JavaScript définit où la variable est accessible et où elle ne l'est pas. Les variables déclarées à l'intérieur d'une fonction ont une portée locale, tandis que les variables déclarées à l'extérieur de toutes les fonctions ont une portée globale. Les variables var ont une portée de fonction, tandis que les variables let et const ont une portée de bloc.

LA MINUTE DE CODE

STYLISER LE DOM

La stylisation du DOM en JavaScript peut être effectuée en utilisant les propriétés CSS des éléments HTML.

Les éléments HTML peuvent être sélectionnés à l'aide des méthodes de sélection du DOM, telles que getElementById(), getElementsByClassName() ou querySelector(). Une fois qu'un élément a été sélectionné, sa propriété style peut être utilisée pour définir les propriétés CSS.

Par exemple, si nous avons un élément avec un ID de "monDiv", nous pouvons le sélectionner en utilisant la méthode getElementById() et modifier sa couleur de fond en utilisant la propriété style.backgroundColor:

```
const monDiv = document.getElementById("monDiv");
monDiv.style.backgroundColor = "red";
```

STYLISER LE DOM

En ce qui concerne les propriétés innerHTML, innerText et textContent, elles sont utilisées pour accéder et modifier le contenu d'un élément HTML. La différence entre ces propriétés est la façon dont elles gèrent le contenu HTML et le texte brut.

La propriété innerHTML permet d'accéder ou de modifier le contenu HTML d'un élément. Elle permet d'ajouter du HTML dans un élément ou de remplacer le HTML existant. Par exemple:

```
const monDiv = document.getElementById("monDiv");
monDiv.innerHTML = "<h1>Bienvenue sur mon site !</h1>";
```

La propriété innerText permet d'accéder ou de modifier le texte brut d'un élément, sans les balises HTML. Elle est souvent utilisée pour récupérer ou modifier le texte d'un élément qui ne contient que du texte. Par exemple:

```
const monParagraphe =
document.getElementById("monParagraphe");
monParagraphe.innerText = "Voici du texte modifié avec
JavaScript !";
```

La propriété textContent fonctionne de la même manière que innerText, mais elle inclut également le texte des éléments HTML imbriqués. Elle est souvent utilisée pour récupérer ou modifier le texte brut d'un élément qui peut contenir des balises HTML imbriquées. Par exemple:

```
const monDiv = document.getElementById("monDiv");
monDiv.textContent = "Voici du texte modifié avec
JavaScript, même s'il y a des <strong>balises HTML</
strong> imbriquées.";
```

LA MINUTE DE CODE

STYLISER LE DOM

replaceWith() est une méthode qui permet de remplacer un élément HTML par un autre élément HTML. Elle peut être utilisée pour modifier dynamiquement la structure du DOM (Document Object Model) en JavaScript.

La méthode replaceWith() est appelée sur l'élément HTML que vous souhaitez remplacer et prend comme argument un ou plusieurs éléments HTML qui vont le remplacer.

Par exemple, si nous avons un élément avec un ID de "monDiv" et que nous souhaitons le remplacer par un élément <p> avec du texte, nous pouvons utiliser la méthode replaceWith() comme suit:
javascript

```
const monDiv = document.getElementById("monDiv");
const monParagraphe = document.createElement("p");
monParagraphe.textContent = "Voici un paragraphe qui
remplace monDiv !";
monDiv.replaceWith(monParagraphe);
```

Ici, nous avons créé un nouvel élément <p> en utilisant la méthode createElement() et avons défini son texte en utilisant la propriété textContent.

Ensuite, nous avons appelé la méthode replaceWith() sur l'élément monDiv pour le remplacer par monParagraphe.

La méthode createElement() est une méthode qui permet de créer un nouvel élément HTML en JavaScript. Elle prend comme argument le nom de l'élément HTML que vous souhaitez créer.

STYLISER LE DOM

Par exemple, si nous souhaitons créer un nouvel élément <p>, nous pouvons utiliser la méthode createElement() comme suit:

```
const monParagraphe = document.createElement("p");
```

Ici, nous avons créé un nouvel élément <p> en utilisant la méthode createElement(). Nous pouvons maintenant utiliser les propriétés de cet élément (comme textContent, style, etc.) pour le personnaliser avant de l'ajouter au DOM en utilisant les méthodes de manipulation du DOM comme appendChild(), insertBefore(), replaceWith(), etc.

En utilisant la méthode replaceWith() et la méthode createElement(), il est possible de modifier dynamiquement la structure et le contenu du DOM en JavaScript.

laminutedecode.io

@LAMINUTEDECODE